AMERICAN UNIVERSITY OF BEIRUT
MECHANICAL ENGINEERING DEPARTMENT

MECH658: INTRODUACTION TO MACHINE LEARNING
FALL 2025

CAN ML COMPUTE COEFFICIENT OF DRAG USING GEOMETRY ONLY?

Done by
Karim Braidi

Submitted to
DR. JOSEPH BAKARJI
DECEMBER 2025

1-Abstract

While Computational Fluid Dynamics (CFD) simulations are highly accurate, they are computationally expensive and time consuming. This project investigates the capability of machine learning models to estimate coefficient of drag (Cd) using geometric and flow-related features derived from the DrivAerNet parametric dataset. Multiple model approaches were trained using different feature spaces that may or may not include flow-related features. Results showed that geometry-only features were insufficient to reliably predict Cd, as reflected by low coefficient of determination R^2. The inclusion of body type categorization significantly improved model performance, highlighting the importance of flow-sensitive information in Cd prediction.

2-Introduction

The regulations on fuel efficiency and carbon emissions are becoming more stringent than ever; this demands more accurate aerodynamic design and analysis. As a result, computational fluid dynamics (CFD) has become indispensable for predicting and optimizing the design of the cars. In addition, high performance vehicles rely heavily on minimizing drag to reduce fuel consumption and increase the performance. To achieve these goals high-fidelity CFD simulations are needed to compete in the automotive market nowadays, whether your goal is to improve fuel efficiency (increase the range for electric vehicles), or high performance. However, CFD is a computationally heavy process where the software must discretize the domain into elements of around 5 millimeters in size, then after that solve partial differential equations numerically for each element to calculate the pressure and velocity fields across the car and finally come up with the coefficient of drag (Cd).

To be more technical, the Cd is obtained through the numerical solution of the Navier-Stokes equations, which describes the conservation of mass and momentum in a viscous flow field around the vehicle body.

$$\rho\left(\frac{\partial u}{\partial t} + u.\nabla u\right) = -\nabla p + \mu\nabla^2 u \qquad (1.1)$$

Where u is the velocity field, p is the pressure, ρ is the air density, μ is the dynamic viscosity. CFD solver numerically approximates these equations using methods such as finite volume or finite volume discretization over a spatially resolved mesh surrounding the vehicle. The resulting flow field allows for the integration of pressure (from drag) and shear stress (skin friction) components over the vehicle surface to obtain the total drag force,

$$F_D = \int \left(pn_x + \tau_{xj}n_j\right)dS \qquad (1.2)$$

From which the nondimensional drag coefficient is derived as

$$C_D = \frac{2F_D}{\rho U_{inf}^2 A_{ref}} \qquad (1.3)$$

Where U_{inf} is the freestream velocity and A_{ref} is the frontal area. This approach, while highly accurate, is computationally expensive and often requires millions of control volumes of turbulence modeling.

Due to this high complexity of CFD, accurate predictions of the drag coefficient remain a major challenge for vehicle design. This behavior has motivated the prediction of Cd using less complicated and expensive methods. One major method to predicting the Cd is Neural Networks (NN), where a user can use a model trained on predicting Cd using geometric features of a car. However, since Cd typically falls between approximately 0.2-0.4, predicting the accurate number posses a challenge where a NN model should predict highly accurate data that is up to the third or fourth decimal point.

Knowing that training such a NN to detect this accurately isn't cheap computationally either, studies turned focus on physics-informed machine learning (PIML), which is an approach that embeds governing physical equations within data driven models to improve prediction accuracy, physical consistency, and interpretability. The following section reviews recent progress in three interconnected areas: (1) large-scale datasets for data-driven aerodynamics, (2) standardized benchmarking frameworks for evaluating AI models, and (3) physics-informed neural CFD surrogates designed to address mean

regression and improve aerodynamic coefficient prediction.

2.1-Literature Review

Recent advancements in data-driven aerodynamic design have been fueled by the emergence of large, high-fidelity datasets. One of the most significant is **DrivAerNet++**, introduced by Elrefaie, Morar, Dai, and Ahmed (2025). The dataset comprises over 8,000 car geometries simulated using the *k–ω SST* turbulence model and includes multimodal data such as 3D meshes, point clouds, flow fields (velocity, pressure, wall shear stress), and aerodynamic coefficients (*Cd*, *Cl*, *Cm*). This model trains a NN to extract features of the car from a STL file using convolutional neural network (CNN), then tabulates them in a csv file with the simulation outcomes (drag coefficients, lift coefficients, and standard deviation for both).

Elrefaie et al. emphasized that while data-driven approaches can drastically accelerate design iteration times, purely statistical models fail to capture essential flow phenomena such as wake structures and vortex interactions. The authors concluded that incorporating physics-based or governing equations into the model architecture or loss function is necessary to achieve reliable and generalizable aerodynamic predictions.

Tangsali et al. (2025) identified a major limitation in most AI aerodynamics is the excessive reliance on the mean squared error (MSE) loss function, because MSE penalizes average numerical differences and often overlooks physically significant deviations such as localized pressure peaks or flow separation. Tangsali et al. (2025) suggests the use of hybrid loss functions that combine statistical error terms (MSE) with physics-based residuals, such as momentum conservation and pressure-Poisson equations.

A study proposed by Alkin et al. (2025), suggests first the use of Anchored-Branched universal physics transformers (AB-UPT), where the geometry encoding, surface prediction, and volume prediction are separated into different branches, improving the surface-volume interactions. Second, it enforces hard physical constraints like the divergence-free property of velocity field ($\nabla \cdot u = 0$). This constraint ensures that the predicted velocity and voracity fields comply with the fundamental laws of fluid mechanics. Alkin et al. (2025) also concluded that removing physics constraints caused the model's prediction to regress toward the mean Cd, demonstrating the essential role of physics-informed NN in such cases.

2.2-Dataset, Feasibility, and Approach

The dataset used in this project is the Elrefaei et al. (2025) dataset (found in the GitHub repository). This dataset was derived from the DriveAerNet++ dataset that included the STL files and CFD simulations of 8000 car designs. In Elrefaei et al. (2025), half of the DriveAERNet++ STL files were used to extract geometric features of the vehicles, particularly sedan type vehicles. Each sample represents a unique variation of the sedan model, where geometric features such as rear window inclination, trunk length, bumper curvature, diffuser angle, etc… are systematically varied. The target variable is the aerodynamic drag coefficient (Cd) obtained through CFD simulations. The dataset contains 4165 samples, and a total of 23 features were extracted using Graphic Convolutional Neural Network. The drag coefficient varies between 0.203 and 0.3, where each Cd is calculated up to 5 decimal points, with the mean of 0.25379 and standard deviation of approximately 0.12.

Because the dataset is composed of geometric parameters, it is well suited for traditional machine learning algorithms like Linear regression, Random Forest, Gradient Boosting, and SVR, in addition to neural networks. However, homogeneity poses a challenge for the generalization. To address this, we will try expanding the dataset through incorporating other types of cars like SUV and hatchback geometries from existing parametric tables, or through developing a CNN to extract geometric features from the STL files provided by DriveAerNet++ database.

Physics informed parameters will be included in a hybrid loss function and as an extra parameter for traditional models. These parameters include the sphericity, wakeness of a vehicle, wake smoothness, and Reynold approximation.

The evaluation metrics used to measure how good the model generalizes are, Mean Squared Error (MSE) to measure how accurate the model is, and Coefficient of Determination (R^2) which indicates the proportion of variance in the actual drag coefficients that is predictable from the model's predictions, with value 1 representing the perfect fit.

3-Experiments

The data was partitioned into 70 % training 15% and validation, 15% testing, using random state. Note that the random state changes every iteration. The parameters mentioned in equation (1.3), simulation parameters, were held constant to ensure ideal conditions. These parameters are $\rho = 1.184 kg\, m^{-3}$, and $U_{inf} = 30 m/s$. Other implicit parameters that affect Cd were also held constant like $\nu = 1.56 \times 10^{-5} m^2 s^{-1}$, where ν is the kinematic viscosity of the fluid. However, Reynolds' number varied slightly between (8.366-10.06)$\times 10^6$, which is extremely turbulent. Note that extremely turbulent conditions imply highly chaotic nature.

Few models were selected to perform this series of experiments. These models vary from linear to extremely non-linear regressions.

Model	R^2 (Train)	R^2 (Test)	MSE (Train)	MSE (Test)
Linear Regression	0.5	0..45	0.00024	0.00024
Random Forest	0.88	0.42	0.00005	0.00025
Gradient Boosting	0.71	0.5	0.00014	0.00023
SVR	0.75	0.41	0.00012	0.00026
Neural Network	0.86	0.39	0.00007	0.00027
Mean (Cd)	0	-0.002079	0.000485	0.000482

Table 1 Baseline Models for Later Comparison

Before discussing the results note that the error in prediction seems small, however that is due to the nature of Cd since it varies from 0.2-0.3 as mentioned in section 1. In addition to that, hyperparameters of classical models were varied to find the best fitting ones. To prove this, we added a mean Cd baseline to show that just by predicting the mean Cd would achieve comparable results, but that would be just a blind guess. Note that R^2 is the coefficient of determination and it doesn't symbolize the residuals squared.

The results show that gradient boosting performed the best when looking at the coefficient of determination and MSE. Notice that by looking at the coefficient of determination, none of the models captures the variance of Cd. Moreover, some models suggest slight overfitting by comparing training and testing values of MSE.

Neural Network performed that worst compared to other models, even though it contained 2 hidden layers with 96 neurons and ReLU activation functions.

Other geometric features could be derived manually and added to the feature space. The derived features were calculated from the original vehicle shape parameters to capture additional physical patterns. The process was to use first order and second order derivatives to retrieve the slope, smoothness, stream-wise variance, and wake characteristics. Each feature is computed per sample, then standardized across the dataset and concatenated with original geometric features. The result of this experiment is documented in table 2.

Model	R^2 (Train)	R^2 (Test)	MSE (Train)	MSE (Test)
Linear Regression	0.5	0..45	0.00023	0.00024
Random Forest	0.92	0.46	0.00003	0.00025
Gradient Boosting	0.67	0.52	0.00016	0.00025
SVR	-0.07	-0.07	0.00051	0.0005
Neural Network	0.86	0.44	0.00007	0.00027

Table 2 Baseline Models with Added Physics Features

The results in table 2 suggest that these added physics features didn't help at all, the only model that showed a slight improvement in R^2 is Random Forest even though it show signs of overfitting in MSE. Moreover, SVR was affected badly missing all the variance as suggested by R^2.

Table 2 shows the highly chaotic and nonlinear nature of Cd in turbulent flow, since even

the neural network (NN) of 704 neurons with various activation functions didn't capture the output.

As a conclusion of table 1 and 2, they clearly suggest that a different approach should be followed to solve this problem, adding a hybrid loss function or using some clustering or dimensionality reduction techniques.

The first intuition on adding a hybrid loss function with imposed physics constraints was to add penalties based on the slopes of the vehicle's curvature, smoothness of the combined features, wakeness of the car, wake smoothness, and Reynold's approximation. We multiplied weights to each penalty, then added them to the loss function, where the loss function chosen for this experiment was the MSE.

To benchmark the results of NN models, two identical models were introduced. The first being a NN with only geometric and geometry derived features, while the second includes physis penalties in the loss function as discussed earlier. The network architecture consists of multiple hidden layers with a high number of neurons to capture complex nonlinear relationships between the geometric and flow-related input features. To enhance training stability and generalization, several optimization techniques were applied: Batch Normalization was used to stabilize the internal activations and accelerate convergence, Dropout was introduced to reduce overfitting by randomly deactivating neurons during training, and the AdamW optimizer was employed for adaptive learning with decoupled weight decay regularization. Additionally, a learning rate scheduler was implemented to gradually reduce the learning rate when validation performance plateaued, and early stopping was used to halt training once the model stopped improving, preventing overfitting. Results of this experiment are documented in table 3.

Model	R^2 (train)	R^2 (test)	MSE (train)	MSE (test)
Deep NN with Phy features	0.802	0.523	0.000096	0.000213
Deep NN with Phy Penalties	0.83	0.522	0.000079	0.000214

Table 3 Benchmarking NN models

NNs with physics penalties demonstrated slight improvements which suggest that with additional architectural tuning and more detailed physics incorporation can enhance the model more to surpass the accuracy of other high-performance models like Gradient Boost (GB).

However, adding weights to the MSE like that they'll iterate to zero since this will minimize the loss function. So, this approach fails when following this intuition, as a solution to this problem, softmaxed weights were added. The following model learns not only the neural network parameters but also adaptive weights for each physics constraint, normalized through a softmax function to prevent them from collapsing to zero.

Model	R^2 (train)	R^2 (test)	MSE (train)	MSE (test)
Deep NN with Physics Softmaxed Penalties	0.86	0.477	0.000063	0.000251

Table 4 Softmaxed Weights in Hybrid Loss Function

Comparing tables 3 and 4 it's clear that the normal deep NN performed better than adding geometry-based penalizations, since the R^2 of the normal NN was slightly greater than the other NN models.

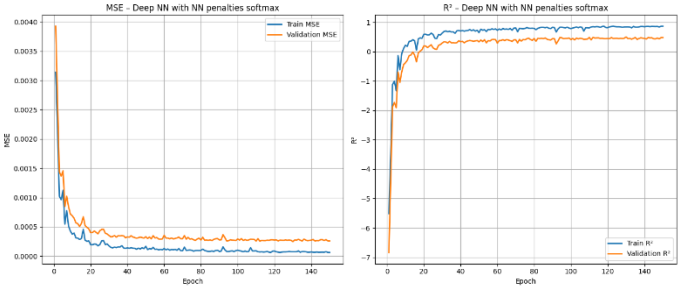

Figure 1: The Performance of the MSE and R^2 using Softmaxed Weights as a Function of Epochs

To help simplify the model Principal Component Analysis (PCA) was applied to the original geometric features in order to reduce dimensionality while preserving 95% of the variance, then augmented these transformed features with geometry-derived features.

Model	R^2 (Train)	R^2 (Test)	MSE (Train)	MSE (Test)
LR (PCA)	0.2	0.2	0.00037	0.00037
RF (PCA)	0.86	0.39	0.00005	0.0003
GB (PCA)	0.56	0.3	0.00023	0.00035
SVR (PCA)	0.58	0.21	0.0002	0.00036

Table 5: Classical Models with PCA

Despite the addition of geometry-derived features, the R^2 values on the test set clearly worsened compared to models with normal features, shown in

table 1. This outcome suggests that PCA transformation may have removed important features that the geometry-based features couldn't capture, or that the geometry-based features themselves aren't strongly predictive with the transformed components. In addition to that, it indicates the need to either a different feature selection strategy or a model capable of learning non-linear interactions, such as NN, may be more effective in Cd prediction.

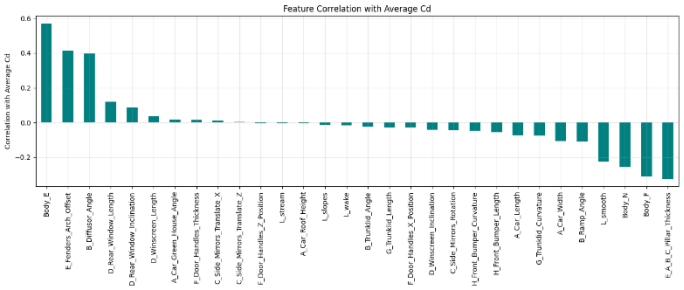

Figure 2: Correlations between Cd and Feature Space

Figure 2 shows that not all geometry-based features have strong correlation with the average Cd. Many geometry and geometric-based features have weak correlations, indicating that geometric features alone might not fully explain the variance of Cd. This was proved by trying both a deep NN, shown in table 3, and dimensionality reduction with classical models, in table 5.

As a solution to this problem an additional categorical feature was added to better represent the latent structural differences within the dataset. Although, all vehicles in the dataset are sedans, they belong to three distinct subcategories (Estateback, Fastback, Notchback). Incorporating this body-type classification will help the models to differentiate between these structural variations and ultimately improve R^2.

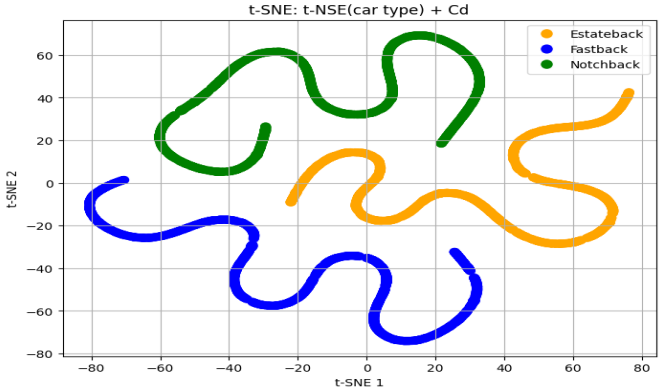

Figure 3: t-SNE Categorizations

The perfect t-SNE lining shown in figure 3 indicates that the data was constructed through iteratively varying geometric features of cars instead of real-life car models.

This feature was derived from the experiment naming convention reported by Elrefaie et al. [1], where the car body type was encoded as letters in the experiments count column. Specifically, the first letter of each experiment identifier indicates the sedan subclass (Estateback, Fastback, or Notchback), which enabled the extraction and inclusion of an explicit body-type classification feature in the dataset.

To include this categorial feature, a string was introduced to the data set, where if the car type was Estateback the added feature would be [1 0 0], if Fastback [0 1 0], and if it was Notchback the added feature would be [0 0 1].

Model	**R^2 (Train)**	**R^2 (Test)**	**MSE (Train)**	**MSE (Test)**
XG Boost + Body Category	0.984	0.843	0.000008	0.000075
RF + Body Category	0.971	0.834	0.000014	0.000079
NN + Body Category	0.925	0.813	0.000037	0.000089
CatBoost + Body + Category	0.966	0.864	0.000017	0.000065

Table 6: ML Models with Categorized Features

Models in table 6 included the models who showed the most potential in previous mentioned experiment, in addition to Cat Boost, which is similar to XG Boost, however it's achieving better results.

All models were able to capture the Cd fully with MSE in the 5th decimal places, and R^2 achieving a nearly 0.9 in Catboost.

The inclusion of body-type categorial features led to noticeable improvement in R^2 values across all evaluated models, as shown in table 6. This drastic improvement of the results indicates that geometric parameters alone were insufficient to fully explain the variability in Cd, as vehicles with similar local geometric features can exhibit different flow structure based on the global shape class.

As further reinforcement that geometry wasn't enough to predict Cd, even though the simulation parameters were held constant, an additional

experiment was conducted in which the lift coefficient (Cl) was incorporated as an input feature with the geometry features. Note that the categorial input feature wasn't added in the following experiment documented in table 7.

Model	R^2 (Train)	R^2 (Test)	MSE (Train)	MSE (Test)
Linear Regression	0.65	0..64	0.000168	0.000169
Random Forest	0.965	0.739	0.00017	0.000125
Gradient Boosting	0.877	0.766	0.000059	0.000112
SVR	0.887	0.712	0.000054	0.000138
Neural Network	0.964	0.76	0.000017	0.000115

Table 7: Baseline Models with Added Coefficient of Lift as Input

Judging by the obtained results in table 7, the addition of Cl provided further evidence that purely geometry-based features were insufficient to fully capture the aerodynamic behavior, as the incorporation of flow-related information improved the model's ability to estimate Cd.

Throughout this study, multiple experiments were conducted to evaluate the predictive potential of different feature representations and model architectures for estimating Cd. Refer to the GitHub repository referenced below for verifying the experiments. The experiments included training classical ML models and deep NN using purely geometry-based features, physics-derived geometric features, hybrid loss functions, and dimensionality reduction representations. In addition, several hyperparameters were varied including learning rates, network depth, number of estimators, tree depth, and regularization settings to ensure the models were not limited to suboptimal tuning. Despite these efforts, geometry-based inputs consistently showed limited improvements in R^2. On the other hand, categorical body-type information and flow related features were introduced, and these additions demonstrated incredible boost in the performance, confirming that geometric parameters alone were insufficient to fully capture the complexity of Cd.

3-Conclusion and Future Work

In this study, we applied geometric features alone to predict the drag coefficient (Cd) of vehicles. While geometric descriptors provided some insight, they proved insufficient for accurate Cd prediction, likely because the mapping from geometry to pressure distribution and shear forces is highly nontrivial. Models relying solely on geometry were unable to fully capture the complex aerodynamic interactions that determine drag.

Future work should focus on incorporating high-fidelity CFD data, including pressure and shear distributions, into predictive models. Leveraging convolutional neural networks (CNNs) or other deep learning architectures on CFD simulated car surfaces could provide richer representations of flow physics, enabling more accurate and generalizable Cd prediction across diverse vehicle designs.

4-References

[1] Elrefaie, M., Morar, F., Dai, A., & Ahmed, F. (2025). DrivAerNet++: A large-scale multimodal car dataset with computational fluid dynamics simulations and deep learning benchmarks. Neural Information Processing Systems (NeurIPS 2024) Datasets and Benchmarks Track. Massachusetts Institute of Technology.

[2] Tangsali, K., Ranade, R., Nabian, M. A., Kamenev, A., Sharpe, P., Ashton, N., Cherukuri, R., & Choudhry, S. (2025). A benchmarking framework for AI models in automotive aerodynamics. NVIDIA Corporation

[3] Alkin, B., Bleeker, M., Kurle, R., Kronlachner, T., Sonnleitner, R., Dorfer, M., & Brandstetter, J. (2025). Scaling neural CFD surrogates for high-fidelity automotive aerodynamics simulations via anchored-branched universal physics transformers. Transactions on Machine Learning Research.

GitHub repository:**https://github.com/KarimBraidi/Drag-Coefficient-Prediction-Using-ML**